blynt

DEVELOPER HANDBOOK & ENGINEERING MANUAL

Architecture Principles • Rapid Onboarding & Local Docker Stack
Monorepo Topology • Strict Ledger Standards • Decimal Math & Prisma ORM
TDD & Invariant Testing • CI/CD & Production Operations • October 2026

Authoritative Engineering Reference & Developer Guide

Standardized Manual for Software Engineers, SREs, QA Engineers & DevOps Specialists

Engineering Stack:	NestJS (Node 20), TypeScript 5, PostgreSQL 16, Prisma, Redis 7, Next.js
Audience:	Software Engineers, Full-Stack Developers, DevOps Engineers, QA Automation
Code Standards:	Zero-Any TypeScript, Decimal(19,4) Math, Strict Hexagonal Domain Isolation
Document Classification:	Enterprise Engineering Standard — Confidential

Document Version: 2.0

Status: Approved & Baselined

Classification: Enterprise Engineering Standard

Target Audience: Backend Engineers, Frontend Engineers, Full-Stack Developers, DevOps/SRE, QA Engineers

---

1. Engineering Philosophy, Principles & System Architecture

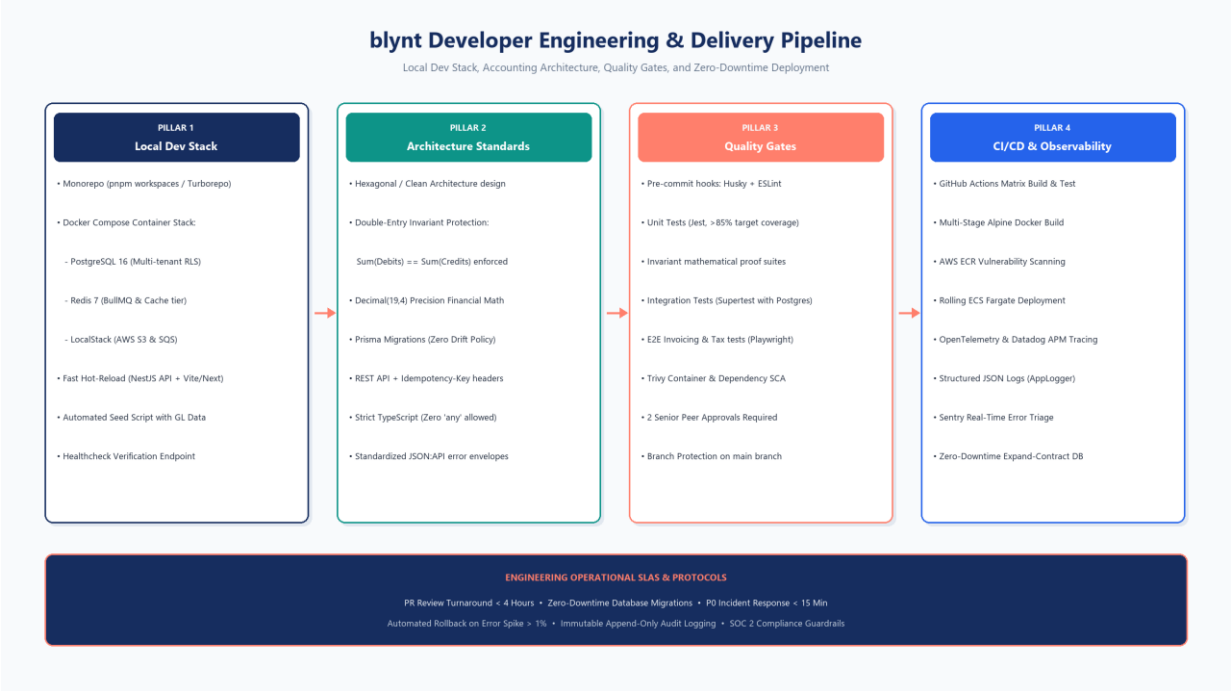


Figure DEV-1: blynt Engineering Architecture & Automated Quality Pipeline

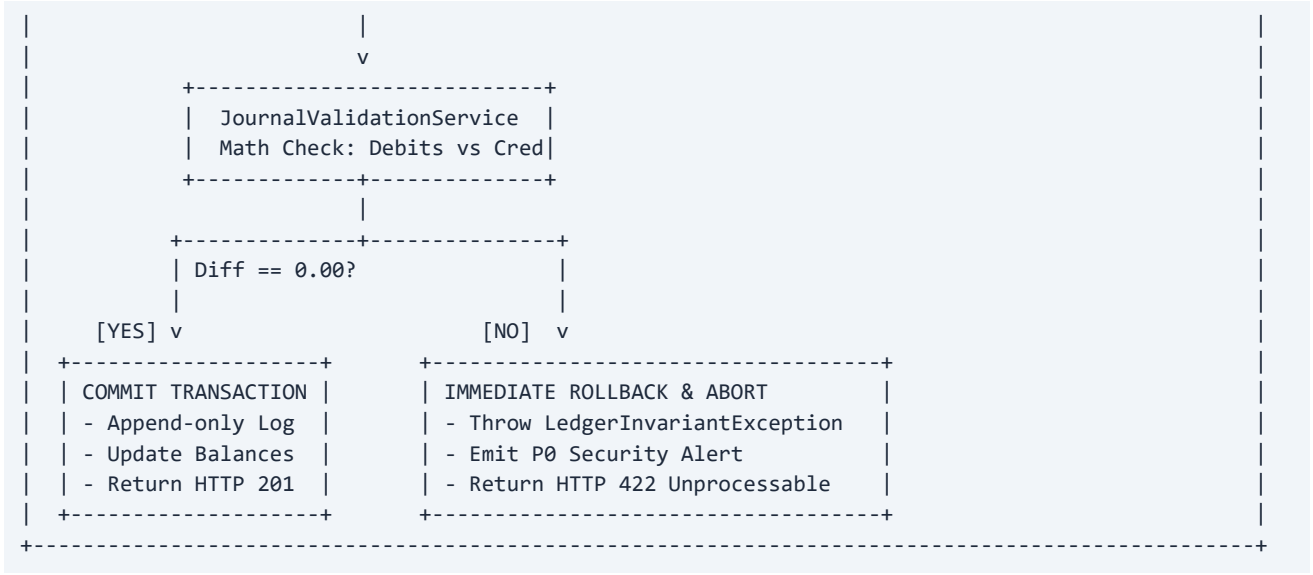
1.1 The Invariant Principle: Mathematical Guarantee of Balanced Books

In financial accounting software, correctness is non-negotiable. Traditional software often treats accounting ledgers as simple transactional tables where balance checks are deferred to end-of-month reconciliations. In **blynt**, we operate on an inviolable foundational principle:

$$\sum_{i=1}^{n} \text{Debit}_i \equiv \sum_{j=1}^{m} \text{Credit}_j \quad \forall \quad \text{Journal Entries}$$

Every financial transaction in blynt is an atomic event. If a mutation attempts to persist even a £0.0001 discrepancy, the entire database transaction is immediately aborted with a `LedgerInvariantViolationException`. There are no temporary "unbalanced" states permitted in persistent storage.

```
+-----------------------------------------------------------------------------------------------+
|                                THE BLYNT POSTING INVARIANT GATE                               |
|                                                                                               |
|   Incoming Journal Payload                                                                    |
|   +-------------------------------------------+                                               |
|   | Lines: [Debit £120.00, Credit £100.00]      |                                               |
|   +---------------------+---------------------+                                               |
```



1.2 Hexagonal Architecture & Clean Domain Boundaries

The backend API is architected according to **Hexagonal (Ports and Adapters) / Clean Architecture** principles. Domain entities and business rules (e.g., Double-Entry Posting, VAT calculation, Bank Rule evaluation) have zero dependencies on frameworks, databases, or external communication protocols.

- **Domain Layer (Core):** Pure TypeScript business logic, math calculations, and domain invariants.
- **Application Layer (Use Cases):** Service orchestrators (`JournalPostingService`, `InvoiceProcessingService`).
- **Adapters / Infrastructure:** NestJS controllers, Prisma repositories, BullMQ message dispatchers, and AWS S3 storage adapters.

1.3 Strict Typing & Zero-Any TypeScript Policy

The blynt codebase enforces TypeScript's strictest compilation settings (`strict: true`, `noImplicitAny: true`, `strictNullChecks: true`, `noUncheckedIndexedAccess: true`).

- **Zero `any` allowed:** The use of `any` will trigger a build-breaking ESLint error in CI.
- **Explicit return types:** All public service methods and controller endpoints must specify explicit return types.
- **Branded types for IDs:** Entity IDs use branded types (e.g., `type TenantId = string & { readonly __brand: unique symbol }`) to prevent accidental argument transpositions.

---

2. Developer Onboarding & Rapid Local Environment Setup

2.1 System Prerequisites

Before initializing the development environment, ensure the following software is installed on your local development workstation:

- **Node.js:** `v20.x LTS` (Recommended via `nvm` or `fnm`)
- **Package Manager:** `pnpm v9.x` (`npm install -g pnpm`)
- **Docker Desktop:** `v24.x` or higher with Docker Compose v2 enabled

- **Git:** `v2.40` or higher
- **PostgreSQL Client (Optional):** `psql` or a GUI tool such as TablePlus / DBeaver

2.2 10-Minute Rapid Onboarding Step-by-Step

Follow this exact sequence to spin up a fully operational, seeded development environment:

```
# 1. Clone the repository
git clone git@github.com:blynt-io/blynt.git
cd blynt

# 2. Install all monorepo dependencies
pnpm install

# 3. Provision environment configuration
cp .env.example .env
cp apps/api/.env.example apps/api/.env
cp apps/web/.env.example apps/web/.env

# 4. Start local backing infrastructure containers
docker compose up -d postgres redis localstack

# 5. Execute database migrations and seed sample chart of accounts
pnpm --filter @blynt/api run prisma:migrate:dev
pnpm --filter @blynt/api run db:seed

# 6. Launch development hot-reload servers
pnpm run dev
```

2.3 Docker Compose Local Topology

The local container topology replicates core AWS production infrastructure:

```
+-------------------------------------------------------------------------------------------+
|                                LOCAL DOCKER COMPOSE TOPOLOGY                              |
+--------------------+---------------+-------------------------------------------------------+
| Container Service  | Exposed Port  | Role & Configuration                                  |
+--------------------+---------------+-------------------------------------------------------+
| postgres-db        | 5432          | PostgreSQL 16 Alpine with custom UUIDv7 extension & RLS    |
| redis-cache        | 6379          | Redis 7 Alpine for session cache, rate-limits & BullMQ jobs |
| localstack-aws     | 4566          | LocalStack emulating AWS S3 (PDF storage) & SQS (Webhooks)  |
+--------------------+---------------+-------------------------------------------------------+
```

2.4 Verifying Local Health

Execute the following verification curl command to validate that the backend NestJS service is fully booted and healthy:

```
curl -X GET http://localhost:3001/api/health
```

Expected JSON Response:

```
{
```

```
  "status": "ok",
  "info": {
    "database": { "status": "up" },
    "redis": { "status": "up" },
    "localstack_s3": { "status": "up" }
  },
  "version": "2.0.0",
  "environment": "development"
}
```

---

3. Codebase Directory Structure & Core Modules Deep Dive

3.1 Monorepo Workspace Directory Map

The repository is managed as a high-performance monorepo utilizing **pnpm workspaces** and **Turborepo**:

```
blynt/
├── apps/
│   ├── api/                          # NestJS Backend API (Port 3001)
│   │   ├── prisma/                   # Prisma schema, migrations, seed data
│   │   └── src/
│   │       ├── accounting-engine/    # Central double-entry invariant & journal services
│   │       ├── auth/                 # JWT, MFA, Argon2id, RBAC guards
│   │       ├── banking/              # Open Banking feeds, bank rules, reconciliation
│   │       ├── common/               # Filters, interceptors, decorators, AppLogger
│   │       ├── invoicing/            # Sales invoices, bills, credit notes, PDF builder
│   │       ├── ledger/               # Chart of Accounts, Trial Balance, P&L, Balance Sheet
│   │       ├── tenant/               # Multi-tenant isolation middleware & RLS policies
│   │       └── vat-mtd/              # HMRC MTD VAT returns & fraud headers
│   └── web/                          # Next.js / React Frontend Dashboard (Port 3000)
│       ├── public/                   # Static assets & brand logo marks
│       └── src/
│           ├── app/                  # App Router pages and layouts
│           ├── components/           # Reusable UI component library (Design System)
│           ├── hooks/                # Custom React data & mutation hooks
│           └── lib/                  # API client, formatting utilities, auth tokens
├── packages/
│   ├── shared/                       # Shared financial math helpers & constants
│   └── types/                        # TypeScript DTOs, API interfaces, schema contracts
├── docker-compose.yml                # Local dev service dependencies
├── package.json                      # Monorepo root manifest
└── pnpm-workspace.yaml               # pnpm workspace definition
```

3.2 Backend Module Responsibilities

- **`accounting-engine`**: The heart of the platform. Houses `JournalValidationService` and `PostingService`. Responsible for debit/credit balancing, concurrency locking, and immutable audit stamping.
- **`tenant`**: Intercepts every inbound HTTP request via `TenantResolutionMiddleware`. Extracts the active `tenant_id` from the verified JWT and configures the PostgreSQL session context (`SET LOCAL app.current_tenant_id = '...'`).

- **`vat-mtd`**: Calculates 9-box VAT returns based on transactional tax codes (Standard 20%, Reduced 5%, Zero 0%, Exempt). Handles OAuth2 token exchange with the UK HMRC API gateway and constructs statutory fraud prevention headers.

---

4. Database Design, Prisma ORM & Persistence Standards

4.1 Schema Modeling Best Practices

To maintain multi-tenant isolation and strict audit compliance, every persistent model in Prisma must adhere to the following rules:

1. **Primary Keys:** Every table uses UUIDv7 primary keys (`@default(dbgenerated("gen_random_uuid()"))` or application-level UUIDv7 generation) to combine chronological sortability with global uniqueness.

2. **Tenant Scoping:** All tenant-scoped entities must include `tenantId String @db.Uuid`. Indexes must lead with `tenantId` (e.g., `@@index([tenantId, createdAt])`).

3. **Audit Timestamps:** Every model must have `createdAt DateTime @default(now())` and `updatedAt DateTime @updatedAt`. Financial ledger rows are **immutable** and omit `updatedAt`.

4.2 Financial Math & Precision: Strict Decimal(19,4) Rule

CRITICAL WARNING: Never use JavaScript `number` or database `FLOAT` / `DOUBLE PRECISION` for currency values. Floating-point arithmetic introduces IEEE 754 rounding errors (e.g., `0.1 + 0.2 === 0.30000000000000004`) which corrupt financial ledgers.

- **Database Representation:** All currency amounts must be stored as `DECIMAL(19, 4)` in PostgreSQL.
- **Application Representation:** Use the `Decimal` class from `decimal.js` or Prisma's built-in `Prisma.Decimal`.

```
// CORRECT: High-precision Decimal calculation
import { Decimal } from 'decimal.js';

const unitPrice = new Decimal('149.99');
const quantity = new Decimal('3');
const taxRate = new Decimal('0.20'); // 20% Standard UK VAT

const subtotal = unitPrice.mul(quantity); // 449.9700
const taxAmount = subtotal.mul(taxRate);   // 89.9940 -> 89.99 rounded
const total = subtotal.plus(taxAmount);     // 539.9640

// INCORRECT: PROHIBITED IN BLYNT CODEBASE
const badTotal = (149.99 * 3) * 1.20; // ERROR: Incurable IEEE 754 precision drift!
```

4.3 Prisma Migration Lifecycle & Zero-Downtime Rule

When adding or altering database columns:

- **Expand-and-Contract Pattern:** Always make additions backward compatible.

1. *Step 1 (Expand):* Add new column as optional (`Nullable`). Deploy application code that writes to both old and new columns.

2. *Step 2 (Migrate):* Backfill existing rows via asynchronous background migration script.

3. *Step 3 (Contract):* Set new column to `NOT NULL` and deprecate/drop the old column.

- **Migration Command:**

```
pnpm --filter @blynt/api run prisma migrate dev --name <descriptive_snake_case_name>
```

---

5. API Design Conventions, DTO Validation & Error Handling

5.1 RESTful Resource Modeling & Idempotency

All API endpoints follow RESTful conventions. Mutations that affect financial ledgers must require an `Idempotency-Key` header:

```
POST /api/v1/invoices HTTP/1.1
Host: api.blynt.io
Authorization: Bearer <JWT_ACCESS_TOKEN>
Idempotency-Key: 7b311352-87ad-4521-968c-ff5dfce5a321
Content-Type: application/json

{
  "customerId": "9a1e0b51-2c1b-410a-8bf7-814d48b11100",
  "issueDate": "2026-10-12",
  "dueDate": "2026-11-12",
  "lines": [
    {
      "description": "Annual Enterprise Accounting Subscription",
      "quantity": 1,
      "unitPrice": "2400.00",
      "taxCode": "STANDARD_20"
    }
  ]
}
```

5.2 Standard Error Response Envelope

All application errors thrown across controllers are caught by the global `HttpExceptionFilter` and formatted into a unified JSON envelope:

```
{
  "statusCode": 422,
  "error": "Unprocessable Entity",
  "errorCode": "LEDGER_INVARIANT_VIOLATION",
  "message": "Debit sum (1420.5000) does not equal Credit sum (1400.0000). Difference: 20.5000",
  "timestamp": "2026-10-12T00:35:12.104Z",
  "path": "/api/v1/journal-entries",
  "traceId": "req_88a91c01ff3941"
}
```

---

6. Automated Testing Strategy, TDD & Invariant Verification

6.1 The blynt Testing Pyramid

Our testing strategy ensures that financial logic is validated through rigorous, multi-tiered testing:

```
                        /\
                       /  \
                      / E2E \          Playwright (Critical Invoicing & VAT flows)
                     /-------\
                    / Integration \    Supertest with real PostgreSQL test container
                   /---------------\
                  /   Unit Tests    \   Jest: >85% coverage on math, posting & invariants
                 /-------------------\
```

6.2 Writing an Invariant Unit Test

Below is an example of an authoritative unit test asserting that the `JournalValidationService` strictly enforces balanced debits and credits:

```
import { Test, TestingModule } from '@nestjs/testing';
import { JournalValidationService } from './journal-validation.service';
import { LedgerInvariantException } from '../common/exceptions';

describe('JournalValidationService (Mathematical Invariants)', () => {
  let service: JournalValidationService;

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      providers: [JournalValidationService],
    }).compile();

    service = module.get<JournalValidationService>(JournalValidationService);
  });

  it('should accept a balanced journal entry', () => {
    const lines = [
      { accountId: 'acc_1', debit: '100.00', credit: '0.00' },
      { accountId: 'acc_2', debit: '0.00', credit: '100.00' },
    ];
    expect(() => service.validateBalanced(lines)).not.toThrow();
  });

  it('should reject an unbalanced journal entry with LedgerInvariantException', () => {
    const unbalancedLines = [
      { accountId: 'acc_1', debit: '100.00', credit: '0.00' },
      { accountId: 'acc_2', debit: '0.00', credit: '99.99' },
    ];
    expect(() => service.validateBalanced(unbalancedLines)).toThrow(LedgerInvariantException);
  });
});
```

---

7. Git Workflow, Code Review & Pull Request Standards

7.1 Trunk-Based Development & Branch Naming

All engineers practice Trunk-Based Development. Feature branches must be short-lived (maximum lifespan of 48 hours).

- **Branch Naming Standard:**
- `feat/<jira-id>-short-description` (e.g., `feat/BLYNT-102-bank-feed-oauth`)
- `fix/<jira-id>-short-description` (e.g., `fix/BLYNT-105-vat-rounding-bug`)
- `refactor/<jira-id>-short-description` (e.g., `refactor/BLYNT-110-decimal-opt`)

7.2 Pull Request Submission Checklist

Before requesting review on a Pull Request, the author must verify:

- [x] Branch is rebased against the latest `main`.
- [x] All unit and integration tests pass locally (`pnpm test`).
- [x] No ESLint or Prettier warnings exist (`pnpm lint`).
- [x] Database migrations (if any) are tested with rollbacks.
- [x] No sensitive credentials or API keys committed.
- [x] Pull Request description references the active Jira ticket.

---

8. CI/CD Pipeline, Deployment & Observability Runbook

8.1 Automated CI Pipeline Stages

Every commit pushed to a pull request triggers the automated GitHub Actions CI pipeline:

```
[ Push to PR ]
      |
      v
+------------------+
| Lint & Formatting| --> ESLint & Prettier check
+--------+---------+
         |
         v
+------------------+
| Type Checking    | --> tsc --noEmit (Zero-Any enforcement)
+--------+---------+
         |
         v
+------------------+
| Unit Tests       | --> Jest unit suites (Math & Invariant verification)
+--------+---------+
         |
         v
+------------------+
| Integration Tests| --> Supertest with ephemeral PostgreSQL container
+--------+---------+
         |
```

```
         v
+------------------+
| Security SCA     | --> Trivy container scan & npm audit
+------------------+
```

8.2 Production Deployment & Observability Runbook

- **Deployment Target:** AWS ECS Fargate running multi-stage Alpine Docker containers.
- **Rolling Updates:** Zero-downtime rolling updates with a minimum healthy percentage of 100% and maximum percentage of 200%.
- **Observability APM:** Datadog agent collects distributed OpenTelemetry traces across API requests.
- **Alert Escalation:**
- **P0 Alert:** Error rate exceeds 1.0% or any `LedgerInvariantException` detected. On-call engineer paged within 5 minutes via PagerDuty.
- **Automatic Rollback:** Triggered if container healthchecks fail for 3 consecutive intervals post-deployment.